

Anwendung von Datenbanksystemen - Musterlösung Kurzfassung

Aufgaben 1-9 · HR-Schema · Stand 13.07.2026 · Vollversion: 26-seitige Musterlösung im Repo

Aufgabe 1 - ER-Modell (Chen-Notation)

1.2 Region/Land/Standort/Abteilung:

Kette von 1:n-Beziehungen: Region –(ist Teil von)– Land –(liegt in)– Standort –(ist angesiedelt in)– Abteilung. Jeweils "1" auf übergeordneter Seite, "n" auf untergeordneter. Attribute: Region(ID,Name), Land(ID,Name), Standort(ID, Straße, PLZ, Stadt, Bundesland), Abteilung(ID,Name).

1.2.2 Diskussion - Abt. an mehreren

Standorten:→ 1:n wird zu n:m ⇒ eigene Verbindungstabelle
abteilung_standort(abteilung_id,standort_id) nötig, da m:n-Beziehungen im relationalen Modell nicht als FK-Spalte abbildbar sind.

Beliebig tiefe geogr. Hierarchie:→ feste Kette (Region→Land→...) kann nicht unterschiedlich tiefe Bäume abbilden.
Lösung: 1 generischer Entitätstyp "Geografische Einheit" (ID,Name,Typ/Ebene) mit rekursiver 1:n-Beziehung "ist Teil von" auf sich selbst.

1.3 Mitarbeiter: neu an Abteilung via "gehört zu" (1:n). Attribute: ID, Vorname, Nachname, E-Mail, Telefon, Einstelldatum, Gehalt, Provisionssatz. **Vorgesetzter:** rekursive 1:n-Beziehung auf Mitarbeiter selbst (Rolle Vorgesetzter=1, unterstellter MA=n).

1.4 Projekte:→ n:m-Beziehung "arbeitet mit" zw. Mitarbeiter/Projekt, da beide Seiten mehrfach. Beziehungsattribute (nur bei n:m erlaubt!): Anteil, Projektleitung. Projekt(ID,Name).

1.5 Jobs:→ 1:n "ist angestellt als" (Job→Mitarbeiter). Job(ID,Titel,+ **abgeleitete** (gestrichelt) Attribute Mindest-/Maximalgehalt, da aus employee.salary berechenbar).

1.6 Beschäftigungsverlauf:→ bezieht sich gleichzeitig auf 3 Entitätstypen (Mitarbeiter,Abteilung,Job), keine 2er-Kombi ist eindeutig ⇒ eigener **schwacher Entitätstyp** (Doppelrand) mit 3× 1:n-Beziehung dahin. Attribute: Start_Datum, End_Datum. Basis für Aufgabe 2.

Aufgabe 2 - Relationales Modell

2.1 Entitätstypen→Relationen: 1:1 Übernahme, Schlüsselattribut wird PK.

```
region(region_id,region_name)
country(country_id,country_name)
location(location_id,street_address,postal_code,
         city,state_province)
department(department_id,department_name)
employee(employee_id,first_name,last_name,email,
         phone_number,hire_date,salary,commission_pct)
job(job_id,job_title,min_salary,max_salary)
project(project_id,project_name)
```

2.2 Beziehungstypen: Regel: **1:n** ⇒ FK-Spalte auf "n"-Seite, **keine** eigene Relation. **n:m** (+mehrstellig/assoziativ) ⇒ eigene Verbindungsrelation, PK = Kombination der FKs.

- ist Teil von: country.region_id→region
- liegt in: location.country_id→country

- ist angesiedelt in: department.location_id→location
- gehört zu: employee.department_id→department
- ist angestellt als: employee.job_id→job
- Vorgesetzter (rekursiv): employee.manager_id → employee.employee_id

```
employee2project(employee_id,project_id,
                 assoc_pct,is_project_lead) --
PK=(employee_id,project_id)
job_history(employee_id,start_date,end_date,
            job_id,department_id) --
PK=(employee_id,start_date)
```

2.3 Vereinfachung:→ nicht weiter nötig – alle 1:n bereits als FK integriert; Verschmelzung nur bei 1:1 sinnvoll (hier keine vorhanden). Ergebnis: genau **9 Relationen**.

2.4 Datentypen (vollständiges HR-Schema):

region(region_id INT PK, region_name VARCHAR25)
country(country_id CHAR2 PK, country_name VARCHAR40, region_id INT FK)
location(location_id INT PK AI, street_address VARCHAR40, postal_code VARCHAR12, city VARCHAR30 NN, state_province VARCHAR25, country_id CHAR2 FK NN)
department(department_id INT PK, department_name VARCHAR30 NN, location_id INT FK)
job(job_id VARCHAR10 PK, job_title VARCHAR35 NN, min_salary DEC(8,0), max_salary DEC(8,0))
employee(employee_id INT PK, first_name VARCHAR20, last_name VARCHAR25 NN, email VARCHAR25 NN, phone_number VARCHAR20, hire_date DATE NN, job_id FK NN, salary DEC(8,2) NN, commission_pct DEC(2,2), manager_id FK(self), department_id FK)
job_history(employee_id FK+PK, start_date DATE PK, end_date DATE NN, job_id FK NN, department_id FK NN)
project(project_id INT PK, project_name VARCHAR50)
employee2project(employee_id FK+PK, project_id FK+PK, assoc_pct DEC(3,2), is_project_lead BOOL)
FK-Pfeil zeigt auf referenzierten PK; manager_id ist Selbstreferenz auf employee.employee_id.

Aufgabe 3 - Normalisierung

3.1.1 Warum nur zusammengesetzte Schlüssel 2NF verletzen:→ 2NF verlangt volle (nicht nur partielle) funktionale Abhängigkeit. Partielle Abh. setzt echte Teilmenge des Schlüssels voraus – bei einwertigem Schlüssel gibt es keine nichttriviale echte Teilmenge ⇒ Verletzung strukturell unmöglich.

3.2 FDs (Rechnung: RNr, KDNr, Name, Wohnort, Pos, Datum, Betrag):

```
RNr → KDNr, Pos, Datum, Betrag
KDNr → Name, Wohnort
(transitiv:) RNr → Name, Wohnort
```

3.3 Verstoß 3NF & Anomalien:→ Name/ Wohnort hängen nur transitiv über KDNr vom Schlüssel RNr ab.

- **Änderungsanomalie:** Umzug ⇒ Wohnort muss in mehreren Zeilen geändert werden
- **Einfügeanomalie:** neuer Kunde ohne Rechnung nicht speicherbar
- **Löschanomalie:** letzte Rechnung löschen ⇒ Kundendaten verloren

Zerlegung: Kunde(KDNr,Name,Wohnort) , Rechnung(RNr,KDNr,Pos,Datum,Betrag)

3.4 BCNF:→ BCNF: jede nichttriviale FD X→Y muss X=Superschlüssel. Beide Relationen aus 3.3 erfüllen dies bereits (KDNr bzw. RNr sind Schlüssel) ⇒ keine weitere Zerlegung nötig.

3.5 Hochschul-Schema:→ Dozent/ Studierende/Vorlesung/Prüfung: einwertiger Schlüssel ⇒ automatisch BCNF.

Kapitel(Modulnr,Überschrift,Inhalt): Inhalt hängt von beiden Teilen ab ⇒ BCNF.
hört(Matnr,Modulnr,Sem): nur Schlüsselattribute ⇒ trivial BCNF.
Raumbelegung(Raum,Tag,Zeit,Sem,Modulnr): Modulnr hängt von voller Kombi ab ⇒ BCNF.
schreibt(Modulnr,PrüfID,Matr,Note): Note hängt von voller Kombi ab ⇒ BCNF.

Ergebnis: gesamtes Schema in BCNF.

3.6 HR-Schema:→ Alle 9 Relationen: einwertiger PK oder Nicht-Schlüsselattribute hängen von voller Schlüsselkombi ab ⇒ **BCNF**. Sonderfälle (Trade-offs, kein echter Verstoß):

- postal_code → city / state_province wäre transitiv (3NF-Verstoß), gilt aber nicht global (viele Länder ohne PLZ, NULL) ⇒ nicht gewertet
- job.min/max_salary sind aus employee.salary abgeleitet – kein BCNF-Verstoß in job selbst, aber relationenübergreifende Redundanz (muss manuell konsistent gehalten werden)

Aufgabe 4 - DDL Teil 1

4.2 Schema:

```
DROP DATABASE IF EXISTS hr; CREATE DATABASE hr; USE hr;
```

DROP IF EXISTS ⇒ Skript beliebig oft wiederholbar.

4.3 Tabellen: CREATE TABLE je Relation aus 2.4 mit PK. NOT NULL bei Pflichtfeldern (Namen, Datumswerte, E-Mail); nullable bei optionalen (commission_pct, manager_id b. CEO, department_id vor Zuordnung). location_id AUTO_INCREMENT.

4.4.1 Fremdschlüssel (ALTER TABLE...ADD FOREIGN KEY):

- country.region_id→region: ON UPDATE CASCADE, ON DELETE RESTRICT
- location.country_id→country: CASCADE/ RESTRICT
- department.location_id→location: CASCADE/RESTRICT
- employee.job_id→job, employee.department_id→department, employee.manager_id→employee: je CASCADE/RESTRICT
- job_history.employee_id/job_id/ department_id: CASCADE/**CASCADE**
- employee2project.employee_id/project_id: CASCADE/**CASCADE**

Prinzip: strukturelle/organisatorische FKs (Geo-Hierarchie, manager_id) = RESTRICT; reine Historien-/Detailtabellen (job_history, employee2project) = CASCADE.

4.4.2 Verständnisfragen:

- RESTRICT bei department_id: Abteilung mit MA nicht löschar, Fehler
- location_id-Update: CASCADE ⇒ automatisch in department übernommen, kein Fehler
- Location mit Depts löschen: RESTRICT ⇒ abgelehnt, erst Depts löschen/verschieben

- MA löschen (ist Manager): RESTRICT verhindert, solange unterstellte MA referenzieren
- MA löschen (kein Manager): erlaubt; job_history+employee2project CASCADE-mitgelöscht
- alles CASCADE + Region löschen: Kettenreaktion Region → Country → Location → Dept → Employee (+rekursiv Manager-Kette) → job_history / employee2project – ganze DB-Teile weg. Deshalb bewusst RESTRICT für Struktur-FKs.

Aufgabe 5 - Constraints

5.1+5.2 CHECK-Constraints:

```
job: CHECK(min_salary>0),
CHECK(max_salary>=min_salary)
employee: CHECK(salary>0),
CHECK(commission_pct>0),
CHECK(email RLIKE '^([a-zA-Z][a-zA-Z0-9])*@[a-zA-Z]+\.[a-z]{2,4}$')
job_history: CHECK(end_date>start_date)
employee2project: CHECK(assoc_pct>0 AND assoc_pct<=1)
location: CHECK(street_address RLIKE '[0-9]')
```

E-Mail-Regex: ^/\$ verankern Anfang/Ende; erster Buchstabe Pflicht; [a-zA-Z0-9]* vor @; nach @ Buchstaben+Punkt; {2,4} TLD-Länge.

5.3 Komplexe (zeilen-/ tabellenübergreifende) Constraints:→ einfacher CHECK prüft nur eigene Zeile ⇒ für Prüfungen über mehrere Zeilen/Tabellen sind **TRIGGER** nötig.

- MA≠eigener Vorgesetzter: noch simpler CHECK(employee_id<>manager_id)
- Gehalt im Jobrahmen: BEFORE INSERT-TRIGGER liest min/max_salary aus job, SIGNAL SQLSTATE '45000' falls außerhalb
- Σassoc_pct pro MA ≤1: BEFORE INSERT-TRIGGER auf employee2project, SUM(assoc_pct) der bisherigen Zeilen + NEW >1 ⇒ SIGNAL (in Beispieldaten bewusst nicht erzwungen, s. 8.4.1)
- job_history-Zeiträume dürfen sich nicht überlappen & start_date≥hire_date: analog per TRIGGER (Vergleich mit vorhandenen Zeilen)

Aufgabe 6 - DML

6.1 INSERT:

```
INSERT INTO region VALUES (1,'Europe');
INSERT INTO country VALUES ('IT','Italy',1);
INSERT INTO employee2project VALUES (186,102,0.70,FALSE);
```

6.2 UPDATE:

```
UPDATE employee SET
phone_number='650.127.3418'
WHERE first_name='Michael' AND
last_name='Rogers';
UPDATE job SET min_salary=4000 WHERE
min_salary<4000;
UPDATE employee SET salary=salary*1.03;
```

6.3 DELETE + CASCADE-Effekt:

```
DELETE FROM employee2project WHERE
assoc_pct<=0.25;
DELETE FROM employee WHERE
first_name='Jonathon'
AND last_name='Taylor';
```

→ job_history(employee_id) hat ON DELETE CASCADE auf employee ⇒ beim Löschen des MA werden automatisch alle zugehörigen job_history-Zeilen mitgelöscht (kein separates DELETE nötig).

6.4 Constraints testen:→ je 1 INSERT gültig + 1 verletzend pro Constraint aus 5.1/5.2, z.B. min_salary=-100 (Fehler), max_salary<min_salary (Fehler), salary<0 (Fehler), end_date<start_date (Fehler), assoc_pct>1 (Fehler), street_address ohne Ziffer (Fehler), email ohne @-Muster (Fehler).

Aufgabe 7 - SELECT Grundlagen

7.1 SELECT country_name FROM country;

7.2 Uhrzeit HH:MM SELECT DATE_FORMAT(NOW(),'%H:%i') FROM DUAL;

7.3 DISTINCT Manager-IDs SELECT DISTINCT manager_id FROM employee WHERE manager_id IS NOT NULL;

7.4 String-Ops (Stadt-Format, NULL→"unbekannt"):

```
SELECT street_address AS Adresse,
IFNULL(CONCAT(country_id,' - ',postal_code,' ',city,' (' ,state_province,')'),'unbekannt') AS Stadt
FROM location;
```

7.5 CEO (kein Vorgesetzter) SELECT * FROM employee WHERE manager_id IS NULL;

7.6 Filter mehrere Kriterien SELECT * FROM job WHERE min_salary>2000 AND LOWER(job_id) LIKE '%clerk%';

7.7 Subquery - Manager finden:

```
SELECT * FROM employee WHERE employee_id IN
(SELECT DISTINCT manager_id FROM employee
WHERE manager_id IS NOT NULL);
```

7.8 Komplex (Manager, Gehalt>10000, vor 1996, Betriebszugehörigkeit, Formatierung):

```
SELECT CONCAT(first_name,' ',last_name) AS Name,

REPLACE(LOWER(email),'@example.com','@...') AS Email,
SUBSTR(phone_number,5) AS Telefon,
CONCAT(DATE_FORMAT(hire_date,'%d.%m.%Y'),' (' ,
ROUND(DATEDIFF('2023-01-01',hire_date)/365,1),' Jahre'))
AS Einstellungsdatum,
CONCAT(ROUND(salary/1000,0),'k EUR') AS Gehalt,
IFNULL(manager_id,'niemand') AS Vorgesetzter
FROM employee
WHERE salary>10000 AND
hire_date<'1996-01-01'
AND employee_id IN (SELECT DISTINCT manager_id
FROM employee WHERE manager_id IS NOT NULL);
```

Aufgabe 8 - SELECT Teil 2

8.1 Sortieren:

```
SELECT last_name,first_name,phone_number
FROM employee
ORDER BY last_name ASC, first_name ASC;
SELECT phone_number,last_name,first_name
FROM employee
ORDER BY phone_number ASC;
```

8.2 Gruppieren:

```
SELECT DATE_FORMAT(hire_date,'%Y') AS Jahr,
COUNT(employee_id) AS Anzahl FROM employee GROUP BY Jahr;
SELECT country_id, COUNT(location_id) AS Anz FROM location
GROUP BY country_id ORDER BY Anz DESC;
```

8.3 Joins (Region/Land/Standort/ Abteilung; Projekt-Zuordnung):

```
SELECT
region_name,country_name,state_province,
CONCAT(postal_code,' ',city,' ',street_address) AS Adresse,
department_name
FROM department
JOIN location ON
department.location_id=location.location_id
JOIN country ON
location.country_id=country.country_id
JOIN region ON
country.region_id=region.region_id;
```

```
SELECT
project_name,first_name,last_name,assoc_pct
FROM project
JOIN employee2project USING(project_id)
JOIN employee USING(employee_id);
```

8.4 GROUP BY über Joins - überlastete MA (>100%):

```
SELECT last_name,first_name,
SUM(assoc_pct) AS Auslastung,
GROUP_CONCAT(project_name SEPARATOR ',') AS Projekte
FROM project
JOIN employee2project USING(project_id)
JOIN employee USING(employee_id)
GROUP BY first_name,last_name
HAVING Auslastung>1;
```

Ergebnis: Baer Hermann, 1.10, "Europe Sales Campaign, US Sales Campaign" – möglich, da Σassoc_pct≤1 in 5.3 bewusst nicht als CHECK erzwungen.

Aufgabe 9 - Mengenoperationen & Views

9.1 UNION - 3 günstigste (≤1996) + 3 günstigste gesamt:

```
(SELECT
employee_id,first_name,last_name,hire_date,
job_id,salary FROM employee
WHERE DATE_FORMAT(hire_date,'%Y')<=1996
ORDER BY salary ASC LIMIT 3)
UNION
(SELECT
employee_id,first_name,last_name,hire_date,
job_id,salary FROM employee ORDER BY
salary ASC LIMIT 3)
ORDER BY salary ASC;
```

UNION entfernt Duplikate ⇒ Ergebnis ≤6 Zeilen.

9.2 View employee_details:

```
CREATE OR REPLACE VIEW employee_details AS
SELECT
employee_id,job_id,manager_id,department_id,
```

location_id,country_id,first_name,last_name,salary,

```
commission_pct,department_name,job_title,city,
state_province,country_name,region_name
FROM employee
JOIN department USING(department_id)
JOIN job USING(job_id)
JOIN location USING(location_id)
JOIN country USING(country_id)
JOIN region USING(region_id);
SELECT * FROM employee_details;
```

USING(...) möglich, da FK-Spalten in beiden Tabellen gleich benannt sind (vermeidet doppelte Spalte im Ergebnis).